Vibe Coding: Building Real Apps *with AI*

Nine hours of class time. One codebase. An app in the hands of Haverford students by December.

WHEN	WHERE	INSTRUCTOR	PREREQUISITES
Thursdays, 6:00–7:30 PM Every other week	Location TBD	Akramjon Abdurakhimov akramjondev@gmail.com	None. Genuinely none.

The premise

Six sessions is not enough time to teach you to program the way it has been taught for forty years. It is enough to teach you to *ship*. You will direct an AI agent through a real codebase — writing the specs, cutting the scope, reviewing what comes back, and owning what goes live.

Session 1 is not “here is the app.” Session 1 is finding out what is actually broken on this campus and proving it with interviews. We expect to land on **HaverTrack** — a dining-hall food and nutrition tracker for Haverford students — because it clears the bar we set that night: a real user (everyone eats), real data (the Dining Center publishes its menus), and a core hard enough to be worth an agent. Auth, a database other people write to, a third-party feed on a schedule, a camera, and an AI call that has to run on a server: five different hard things in one app.

The whole class ships one app. Teams of three or four own a feature area of the same repository, which means you will merge into other people’s work and they will merge into yours. That friction is the curriculum.

How the class runs

Studio, not lecture. Each session is roughly forty minutes of new material and fifty minutes of building with help in the room.

You decide, the agent types

The bottleneck was never syntax. It is knowing what to build, spotting a confidently wrong answer, and cutting scope before it cuts you.

Nothing ships you can’t explain

If it is in a pull request with your name on it, you can walk me through what it does and why it is there. I will ask.

The work is between sessions

Budget four to six hours per two-week gap. Class time is for unblocking, reviewing, and what only works in a room together.

Real users are watching

Every decision gets tested on a student who did not build it, does not care how hard it was, and will close the app in nine seconds.

What we build on

You need to know none of this on day one. You will have used all of it by December.

`React Native + Expo SDK 57` `Expo Router` `TypeScript` `Supabase Postgres` `Row Level Security` `Supabase Auth` `Edge Functions` `TanStack Query` `Zustand` `Zod` `Git + GitHub` `GitHub Actions` `Claude Code` `MCP servers` `EAS Build`

Calendar

Every other Thursday, September 3 to December 10. Two Thursdays land on College breaks; a Break Sprint replaces each.

DATE		SESSION
Thu Sep 3	01	**Find the problem worth building** — agent capabilities and failure modes, problem filters, interviewing, campus inventory, teams
Thu Sep 17	02	**From problem to spec to first merge** — writing a buildable spec, HaverTrack's architecture, Git and pull requests
Thu Oct 1	03	**Data, auth, and who is allowed to see what** — modeling, SQL, migrations, Row Level Security, email auth
Thu Oct 15	—	Fall Break · no class · **Break Sprint A**
Thu Oct 29	04	**APIs, secrets, and things that fail** — the menu sync, scheduled jobs, Edge Functions, camera, failure states
Thu Nov 12	05	**Design, and testing on people who don't care** — UI/UX fundamentals, design tokens, live usability testing
Thu Nov 26	—	Thanksgiving Break · no class · **Break Sprint B**
Thu Dec 10	06	**Ship day** — pre-flight checklist, builds and release, operating a live app, demos, retro

Break Sprints

Two Thursdays fall on College breaks. There is no class and no office hours those weeks, and each sits inside a four-week gap between sessions — these assignments are what keep the app from going cold.

Break Sprint A · Fall Break week of Oct 15

No class and no office hours that week, and a four-week gap between sessions — this is what keeps the app from going cold.

- Review another team's spec against the rubric and file three specific GitHub issues on their feature area. Not "this is confusing" — what breaks, and for whom.
- Async standup in the class channel by that Friday: shipped, stuck, next.

Break Sprint B · Thanksgiving week of Nov 26

Two weeks from launch, with your beta group using the app over the break — the best bug reports you will get all semester arrive that week.

- Solo ship: one small, self-contained improvement, spec'd and merged without your team's help.
- Three hundred words on something the agent got wrong this semester and how you caught it. Graded on the catching.

01 Find the problem worth building

Thu Sep 3 · 6:00–7:30

Nobody writes code tonight. That is deliberate.

6:00–6:15 The deal

- What we are shipping, who uses it, and what “done” means on December 10
- How the six sessions map onto the build, and where the real hours go
- The one rule: you can explain everything you ship

6:15–6:40 What the agent is, and what it isn’t

- Live: Claude Code takes a small feature from request to working code, narrated
- Where it genuinely excels — unfamiliar APIs, boilerplate, refactors across many files, reading an error message you have never seen
- **Failure mode 1: stale versions.** It will write Expo SDK 49 code into our SDK 57 project. This is why our repo’s first instruction to the agent is to read the versioned docs before writing anything
- **Failure mode 2: invention.** Functions, props and endpoints that do not exist, delivered with total confidence
- **Failure mode 3: silent scope creep.** It edits four files you did not ask about
- Plan before code: review the plan, not the diff. Watching a wrong plan get built is the expensive way to learn this
- Exercise: we ask it for something subtly wrong and race to catch it

6:40–7:05 Finding a problem that is actually real

- **The three filters.** Frequency: daily beats once a semester. Pain: mildly annoying every day beats catastrophic and rare. Reachability: can you put this in front of fifty Haverford students by December?
- “An app for X” is not a problem statement. The format we use: *[who] can’t [do what] because [why], so instead they [current workaround]*
- The workaround is the evidence. If nobody has one, nobody has the problem
- Interviewing for past behavior, not future intentions: “Tell me about the last time…” never “Would you use…”
- Five questions that work, and three that poison your data: “Would you use an app that…”, “Don’t you hate it when…”, “What features do you want?”
- How to take notes: verbatim quotes, not your summary of what they meant

7:05–7:25 The campus problem inventory

- Silent brainstorm, ten minutes, one problem per card — no discussion, no filtering
- Cluster on the board, vote, then rank the top ten against the three filters
- Where we expect to land: dining. Students eat at the DC two or three times a day, the menus are published but not usable as nutrition, and nobody can answer “what did I actually eat today?”
- Stating HaverTrack v1 out loud, with its out-of-scope list, before anyone opens an editor
- Why it is a good teaching project: sign-in, a shared database, a third-party feed on a schedule, a camera, and a model call that must run on a server

7:25–7:30 Teams and feature areas

- **Log & search** — today’s DC menu, dish search, favorites, quick add
- **Scan** — camera, barcode lookup, photo becomes an editable plate
- **Progress** — daily totals, weight history, streaks, insights
- **Account & onboarding** — sign-up, goal setting, personal details, deleting your data

YOU LEAVE WITH

A team, a claimed feature area, and a problem you can defend in one sentence.

INDEPENDENT WORK DUE SEP 17

1. **Five interviews**, ten minutes each, with students who are not in this class. Submit verbatim notes with at least three direct quotes per interview. Ask about the last time, never the next time.
2. **Setup checklist complete** (back page). HaverTrack running on your own phone — screenshot as proof.
3. **One Claude Code session** on any project at all. One paragraph: the first thing it got wrong, and how you noticed.

02 From problem to spec to first merge

Thu Sep 17 · 6:00–7:30

Everyone leaves with code merged into a repository other people depend on.

6:00–6:20 Interview debrief

- What each team actually heard, and where the teams contradict each other
- The chain we will use all semester: **quote → need → feature → the cut**
- Which of your assumptions died this week. Name one out loud

6:20–6:45 Writing a spec an agent can build from

- The seven sections: problem, users, user stories, **out of scope**, data, screens, done criteria
- User story format — and why “as a user I want a nice UI” is not one
- Out-of-scope is the most important section in the document. Everything you did not write there, you have implicitly promised
- Acceptance criteria that can actually be checked: “the totals row updates within one second of adding an item,” not “it feels fast”
- How much detail an agent needs: name the files, name the tables, name the screens. Vague in, vague out
- Decomposition: turning one feature into four tasks that four people can do in parallel without colliding
- Live: we take a vague line from a real spec and rewrite it into something buildable, together

6:45–7:10 The architecture of HaverTrack

- Drawn on the board, then mapped onto folders you can open
- **The phone app** — React Native and Expo; screens live in `src/app`, and the folder structure *is* the navigation (file-based routing)
- **The database** — Supabase Postgres, thirteen tables including `profiles`, `meal_logs`, `meal_log_items`, `menu_items`, `weight_entries`
- **Auth** — Supabase Auth, email address plus a six-digit code
- **The menu feed** — a scheduled GitHub Action calls the Nutrislice API three times a day and writes `menu_items`
- **Server-side work** — Edge Functions: `analyze-photo`, `estimate-goals`, `delete-account`
- The question to ask about every new piece of work: *does this need a secret?* If yes, it cannot run on the phone. That single question decides most of the architecture
- State, briefly: what TanStack Query caches from the server, what Zustand holds on the device, and why they are not the same thing

7:10–7:30 Git, and your first pull request

- The five commands you will use all semester: `clone`, `checkout -b`, `commit`, `push`, and opening a PR
- Branch naming so four teams can work at once: `feature/log-search-empty-state`
- What a good pull request description contains — including what you asked the agent for and what you changed by hand
- Reviewing someone else’s PR: does it do what it says, does it break anything of mine, can I explain it
- We create a merge conflict on purpose and resolve it together, so the first one you meet alone is not a crisis
- Live: every student ships a visible change on a branch and opens a PR before leaving the room

YOU LEAVE WITH

A merged pull request in a codebase other people depend on.

INDEPENDENT WORK DUE OCT 1

1. **Team spec**, one to two pages, committed as `docs/specs/<feature>.md` and merged by pull request. It must contain an out-of-scope list you are willing to be held to.
2. **One more merged PR each**, plus one review of a teammate’s.
3. **Read your feature’s existing screens** in `src/app`. Bring three questions about code you do not understand.

03 Data, auth, and who is allowed to see what

Thu Oct 1 · 6:00–7:30

The night we read another student's food log on purpose, then make it impossible.

6:00–6:20 Spec review

- Two teams present; the class hunts for what is missing from the out-of-scope list
- Where a spec was too vague for the agent, and what it produced instead

6:20–6:50 Modeling the data

- Tables, rows, columns, primary keys, foreign keys — taught on our actual schema, on screen
- Worked example: a logged meal is one row in `meal_logs` (whose, what date, which meal period) with child rows in `meal_log_items` (which dish, what portion). Why that is two tables and not one
- The `user_id` column that appears on nearly every table, and what it is really for
- Nullable versus not-null, defaults, and timestamps that let you ask "what changed since Tuesday"
- Indexes: why `idx_meal_logs_user_date` exists, and the trigram index that makes typo-tolerant dish search possible
- **The six SQL statements** you need this semester: `select`, `insert`, `update`, `delete`, `join`, `where`. There is no seventh

6:50–7:10 Migrations

- Schema as version-controlled files in `supabase/migrations`, named by timestamp so they apply in order
- Running them with `npx tsx scripts/migrate.ts`; what ran is recorded in `schema_migrations`, so re-running is safe
- Why we never add a column in the dashboard: your database and mine silently diverge, and the bug surfaces on someone else's laptop a week later
- Live: add a column with the agent, write the migration, run it, watch it land in the table
- What to do when a migration is wrong — write a new one forward, never edit one that has already run

7:10–7:30 Row Level Security, and signing in

- The idea: the database itself refuses to return rows that are not yours. Not a check in the app — a rule in the database, enforced even if the app is wrong
- Reading a real policy line by line: *"Users can manage own meal logs"*, `using (auth.uid() = user_id)`
- **Live demonstration:** we switch a policy off and read one account's food log from another phone. Then we switch it back on and watch the same request return nothing
- Shared-but-protected data: `menu_items` is readable by anyone signed in and writable by nobody — the sync job writes it with different credentials
- The auth flow end to end: sign up, receive a six-digit code, confirm, hold a session. The email template must include the token or the code is never generated — the single most common setup failure in this stack
- Server-owned columns: `profiles.role` is protected by a database trigger, because a client that can promote itself to admin is not a security model

YOU LEAVE WITH

Your feature's tables migrated, with policies that hold when the class attacks them.

INDEPENDENT WORK · FOUR WEEKS, FALL BREAK IN THE MIDDLE DUE OCT 29

1. **Schema and RLS policies** for your feature area, merged.
2. **Prove it holds.** Sign in as a second account and show your query returns none of the first account's rows. Screenshot in the pull request.
3. **Bring one query** your screen needs and cannot yet write.
4. **Break Sprint A** over Fall Break — see page 2.

04 APIs, secrets, and things that fail

Thu Oct 29 · 6:00–7:30

Where the app stops being a demo and starts depending on the outside world.

6:00–6:15 Security check-in

- Teams attempt to read each other's data. Anything that leaks gets fixed tonight
- Two minutes each: the query you could not write, answered

6:15–6:40 What an API is

- Request, response, JSON, headers — in ten minutes, with a real one open
- Status codes and what each one means you should *do*: `200`, `401`, `404`, `429`, `500`
- Live: we call the Nutrislice endpoint from the terminal and read the raw JSON that becomes tonight's dinner list
- **Validation at the boundary.** Our Zod schemas exist because a third-party feed changes shape without telling you. What the app does when `calories` comes back `null` — because it does

6:40–7:00 The menu sync, opened up

- The pipeline, top to bottom: a scheduled GitHub Action fetches the DC menus, parses them, upserts into `menu_items`, and caches a copy in the repo
- Why the cron runs at 5:30 AM, 11:00 AM and 4:30 PM Eastern — before each meal, not on demand. Scheduling as a product decision
- **Upsert versus insert:** running the same sync twice must not double tonight's menu. Idempotence, without the word
- Secrets in CI: `DATABASE_URL` lives in GitHub Actions secrets and never in the repository. What to do the day one leaks
- What students see when the sync did not run, and why that is a design question, not a bug

7:00–7:15 Secrets and Edge Functions

- **The rule.** Anything shipped inside a phone app can be read by anyone who has the app. Variables prefixed `EXPO_PUBLIC_` are public by definition. There is no clever way around this
- So the photo scan cannot call a model API from the phone — the key would ship to every user
- What an Edge Function is: a small server, deployed separately, that holds the key and does exactly one job. Ours are `analyze-photo`, `estimate-goals` and `delete-account`
- Deploying one, setting its secrets, reading its logs. A `401` from an anonymous `curl` means it is alive and refusing you — the healthy answer
- The service role key: what it bypasses, why it exists, and why it never leaves the server

7:15–7:30 The camera, and designing for failure

- Camera and barcode scanning need a real device — the simulator has no camera, and browsers only allow it over HTTPS
- The scan path: photo → Edge Function → a list of dishes with portions you can edit before saving. Why the user always gets the last word
- Caching lookups in `barcode_cache`: politeness to someone else's API, and speed for the second student who scans the same granola bar
- **The four-states drill.** For every screen you own, name what appears when it is empty, loading, offline, and erroring. "It won't happen" is not one of the four
- Graceful degradation: when the scan service is unavailable the app estimates on-device instead of showing an error page

YOU LEAVE WITH

Your feature working end to end against live data, not fixtures.

INDEPENDENT WORK DUE NOV 12

1. **Your feature works against real data**, merged. Fixtures are for tests, not for demos.
2. **All four states** implemented on every screen you own.
3. **Break it deliberately.** Airplane mode, a slow network, a missing field, a very long dish name. File what you find — you are graded on finding, not on nothing being wrong.

05 Design, and testing on people who don’t care

Thu Nov 12 · 6:00–7:30

The night you are forbidden from explaining your own app.

6:00–6:25

UI/UX fundamentals

- **Visual hierarchy** — what the eye lands on first, and controlling it with size, weight, and space rather than color and boxes
- **Affordance** — a thing must look like what it does. If it is tappable it must look tappable, and if it is not, it must not
- **Touch targets** — 44 points minimum. Why your beautiful small icon button fails on a real thumb in a loud dining hall
- **Accessibility** — text contrast ratios, dynamic type, screen-reader labels on icon-only buttons, and never encoding meaning in color alone
- **The four states as design** — an empty state is the best onboarding screen you will ever get for free, and most apps waste it
- Motion and haptics with a purpose: confirming an action, not decorating one
- Writing in the interface: buttons say what happens, errors say what to do next

6:25–6:45

Design systems

- Tokens over values: colors, spacing and type live in `src/constants/theme.ts`, and no screen hardcodes a hex code
- Shared components — when to extract one, and when extracting one makes everything worse
- Light and dark as one system with two sets of values, not two designs
- One type scale, three weights, and the discipline to stop there
- Consistency beats taste. It is also far cheaper to review, which is why the whole class benefits

6:45–7:00

Directing an agent on design

- Screenshots in, critique out: paste the screen and ask what is wrong with it *before* asking for a fix
- Describing changes as intent — “this row should read as secondary” — rather than as pixels
- Iterating visually: change one thing, screenshot, compare, keep or revert
- Where the agent is genuinely weak at design and the decision has to be yours

7:00–7:30

Usability testing, live

- The method: five users, one task, no explanation, no rescuing. Five is enough — the same problems repeat
- What you say: “Log what you ate at lunch today.” Then nothing. Silence is the instrument
- What to record: where they hesitate, what they tap that is not a button, what they say out loud, where they give up
- **The rule you will hate:** you may not explain your own app. Every explanation you give in a test is a bug you are hiding from yourself
- Live: teams swap phones and test cold on each other, then on someone from outside the class if we can find one
- Debrief: each team names its three worst failures out loud, in the tester’s words rather than their own

YOU LEAVE WITH

Your three worst usability failures, in writing, in someone else’s words.

INDEPENDENT WORK — DUE DEC 10

1. **Fix all three.** Each fix cites the observation it came from in the pull request.
2. **Full QA pass** on a real device: every screen, every state, both themes, a small phone and a large one.
3. **Soft launch to ten students** outside the class. Sit with two of them while they use it and say nothing.
4. **Release-blocking bug list**, agreed with your team and filed by Dec 8. Everything not on it is not shipping.
5. **Break Sprint B** over Thanksgiving — see page 2.

06 Ship day

Thu Dec 10 · 6:00–7:30

The release is cut in the room, in front of everyone.

6:00–6:20 Pre-flight

- The checklist we run before every release: `npx tsc --noEmit` for types, `npx expo export` to prove the bundle actually builds, migrations applied, Edge Functions deployed, environment variables present in every environment
- The three environments — your laptop, CI, production — and the class of bug that only exists because they differ
- Reading the release-blocking bug list out loud and cutting everything that does not ship tonight. **Cutting is the skill.** Shipping is what happens after you cut

6:20–6:45 Deploying

- What Expo Go could do for us all semester, and what it could never do
- What a real build gives you, and what EAS Build does with it
- TestFlight and the web build; version numbers, and why they matter the moment two versions exist in the wild
- Live: we cut the release together, on the projector, mistakes included
- Rollback: exactly what you do when the release is broken at 11 PM and forty people have it

6:45–7:00 Operating something real

- Monitoring: how you find out it is broken before a user tells you
- Support: where bug reports land, who reads them, and how fast you answer
- Privacy in practice — what we collect, why each field earns its place, and the `delete-account` function that genuinely removes it
- The maintenance question, answered tonight: who is on call in January, and how the next cohort picks this up

7:00–7:25 Demos

- Five minutes per team. A real phone, real data, no slides
- Each team shows one thing they shipped, one thing they cut, and one thing that broke
- Students from outside the class install it in the room

7:25–7:30 Retro

- Where the agent carried you, honestly
- Every place you had to actually know something — this is the list that matters
- What you would tell the next cohort in one sentence

> **YOU LEAVE WITH**
>
> An app other people use, and a merge history with your name on it.

Setup checklist

Complete before Session 2. If any step takes more than thirty minutes, stop and bring it to the setup clinic — do not lose a Saturday to an install error.

- [ ] **A laptop you can install software on**
  macOS or Windows. A Chromebook or a locked-down machine will not work for this class.
- [ ] **A GitHub account**
  Send me the username so I can add you to the repository.
- [ ] **Node.js 20 or newer, and Git**
  Verify with `node --version` and `git --version`.
- [ ] **Claude Code, installed and signed in**
  Open it once and ask it something. That is assignment three.
- [ ] **Expo Go on your phone**
  iOS or Android. Session 4's camera work needs a real device — simulators have no camera.
- [ ] **A Supabase account**
  Free tier, using an email address you actually check.
- [ ] **The app running on your phone**
  Clone the repo, `npm install`, `npx expo start`, scan the QR code. Green light: HaverTrack opens on your phone.

Assessment

Weighted toward what you shipped and what you unblocked, because that is what the class is for.

Problem work & interviews Session 1 output and five real conversations, with quotes	10%
Team spec Scoped, honest about what is cut, and specific enough for an agent to act on	15%
Merged pull requests Individual. Reviewing a teammate's work counts toward this	30%
Usability pass & fixes The three failures found, and what you did about each	15%
Launch, demo & retro It works on a stranger's phone, and you can explain how	20%
Break Sprints & unblocking others Standups, reviews, and answering someone else's question	10%

The grade is a proxy. The real test is whether Haverford students are still opening the app in February.

Policies

Using AI

- **Use it for everything.** That is the course, not a loophole in it.
- You may not ship code you cannot explain — not “roughly,” but what it does and why it is there.
- A confident wrong answer is the most expensive thing in this class. Verify against the running app, the database, and the real docs.
- Your PR description says what you asked for and what you changed by hand.
- Specs and retros may be drafted with AI. The argument has to be one you can defend out loud.

Attendance & help

- Six sessions is nine hours. Missing one costs a sixth of the class — tell me in advance and I will get you a catch-up plan.
- Weekly office hours, time TBD, plus a setup clinic thirty minutes before Session 2.
- The class channel beats email. Ask there and someone answers before I do.
- Stuck for over an hour on the same error is not persistence. Post it.

Real users, real responsibility

- HaverTrack holds other students’ food logs and body weight. That is sensitive data about people you will see at lunch.
- Collect the minimum. If no screen uses a field, it does not go in the table.
- Row Level Security is not a later problem.
- No real user data in screenshots, demos, or your downloads folder. Test accounts for testing, seed data for demos.
- Every user can delete their account and everything in it. We ship that; we do not promise it.

What this class is not

- Not a substitute for a computer science degree. You will not leave able to write a compiler, and that is fine.
- Not a survey of AI tools. One workflow, deeply, on one real codebase.
- Not a hackathon. Anyone can build a demo in a weekend; almost nobody keeps one running for a semester.

Three of the things you will do this semester — write a database migration, deploy a server-side function, cut a release build — most computer science majors do not touch until their third year.

You will do them in weeks five, nine, and fifteen, with an agent doing the typing and you doing the deciding. The syntax was never the hard part. Deciding what to build, and being honest about whether it works, always was.